Université de Strasbourg

IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

Introduction aux systèmes d'exploitation et à leur fonctionnement

CM2 : Flux de redirections et Processus

Ali Ayadi

ali.ayadi@unistra.fr

BUT Informatique - S11

2026–2027

Plan

1. Flux de redirections
2. Processus
3. Commandes incontournables

Plan

1. Flux de redirections
2. Processus
3. Commandes incontournables

Comportement par défaut d'une commande

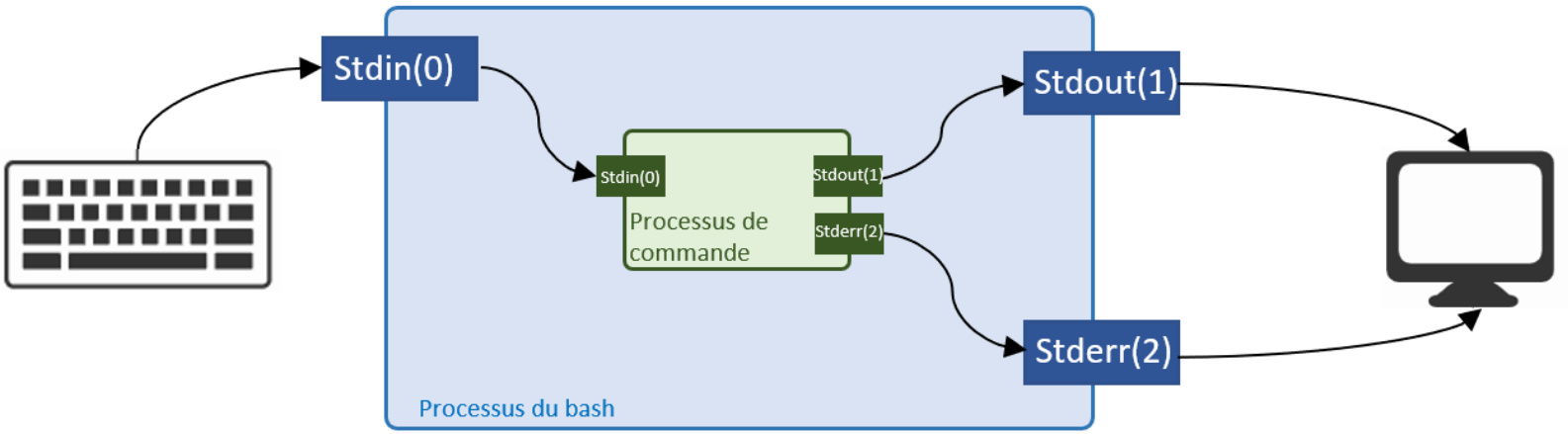
Lorsque l'on exécute une commande, ses données (entrées/sorties)

- sont lues sur l'entrée standard (le **clavier**)
- sont écrites sur la
  - sortie standard (sorties normales)
  - sortie d'erreur standard (messages d'erreur)

  à l'**écran**, dans les deux cas

⇒ On dit qu'une commande est associée à **trois flux** :

1. L'entrée standard (ou "input" ou "stdin") qui correspond à l'entrée 0
2. La sortie standard (ou "output" ou "stdout") qui correspond à l'entrée 1
3. La sortie d'erreur standard ("stderr") qui correspond à l'entrée 2



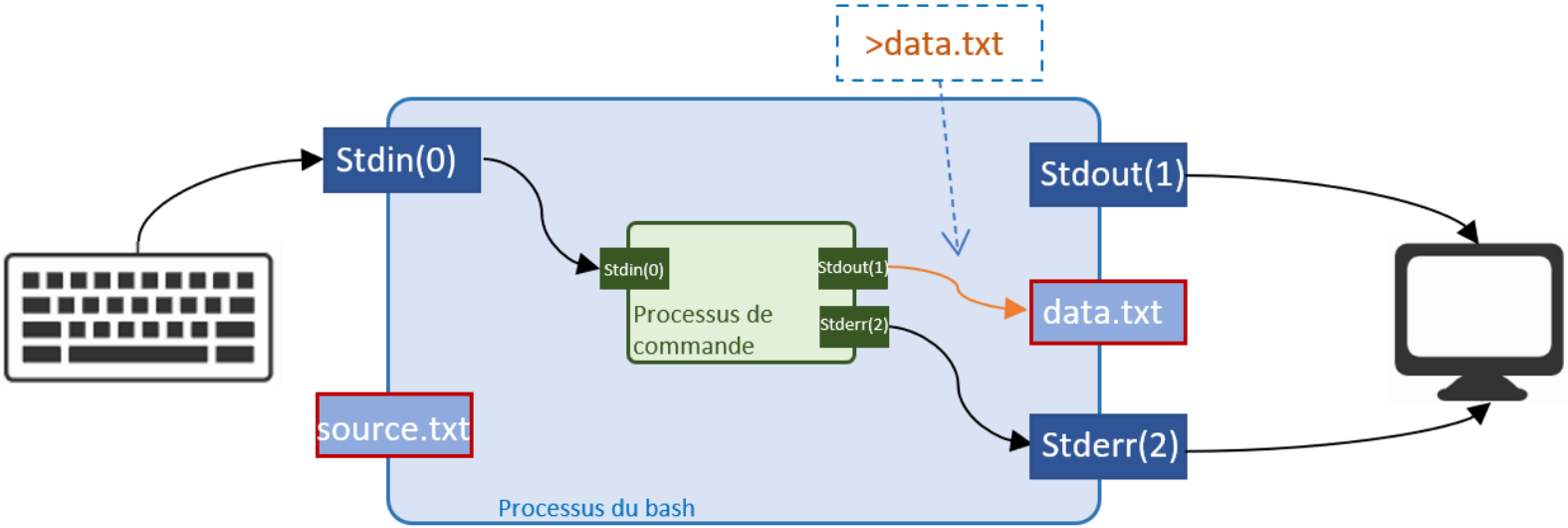
Définition la redirection des entrées/sorties d'une commande

Rediriger

- l'entrée standard signifie ne pas lire depuis le clavier, mais depuis
  - un fichier
  - les sorties d'une autre commande
- une sortie standard signifie ne pas écrire à l'écran, mais vers
  - un fichier
  - les entrées d'une autre commande

Redirection de la sortie standard

On peut rediriger la sortie standard d'une commande vers un fichier avec >.
Le résultat de la commande sera placé dans le **fichier** au lieu de s'afficher sur l'écran.



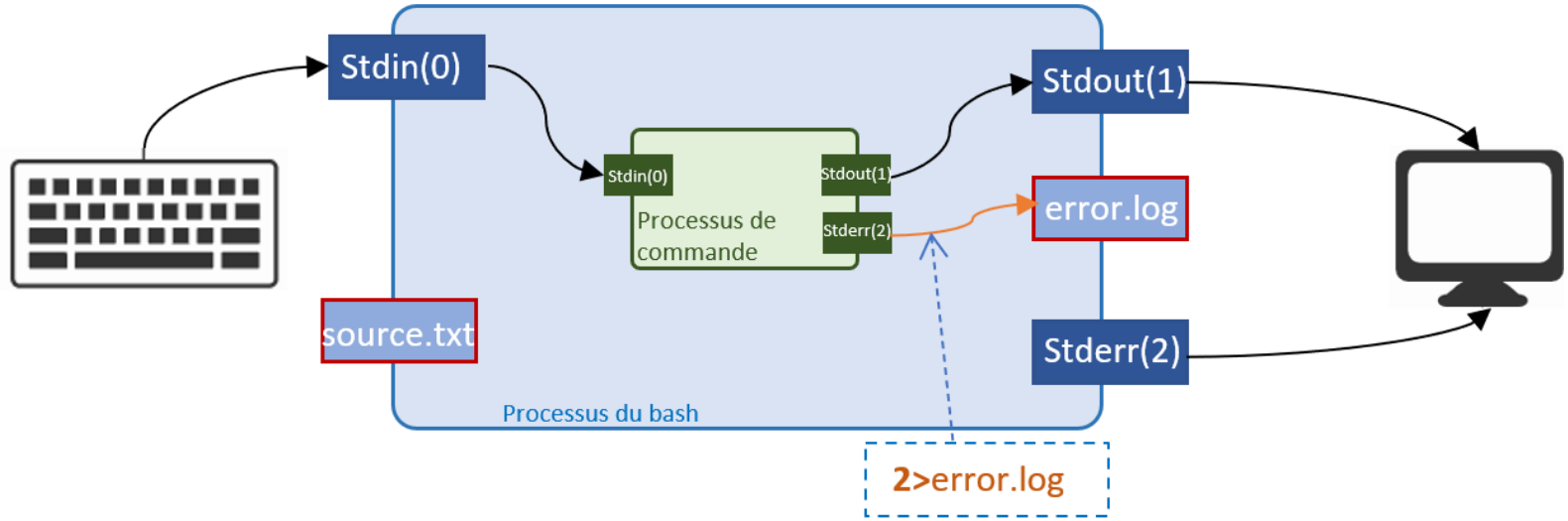
Exemple :

```
ali@DESKTOP-A3SUPDN:~$ echo coucou tous
coucou tous
ali@DESKTOP-A3SUPDN:~$ echo coucou tous >Bonjour.txt
ali@DESKTOP-A3SUPDN:~$ cat Bonjour.txt
coucou tous
```

Attention : On veut parfois ajouter la sortie d'une commande à un fichier, sans effacer ce qui précède. Pour cela on utilise l'opérateur >>
Si le fichier n'existe pas, il est automatiquement créé.

Redirection de la sortie d'erreur

Pour rediriger la sortie d'erreur standard vers un fichier, on utilise l'opérateur 2>
On peut concaténer la sortie d'erreur d'une commande au contenu d'un fichier déjà existant, avec l'opérateur 2>>



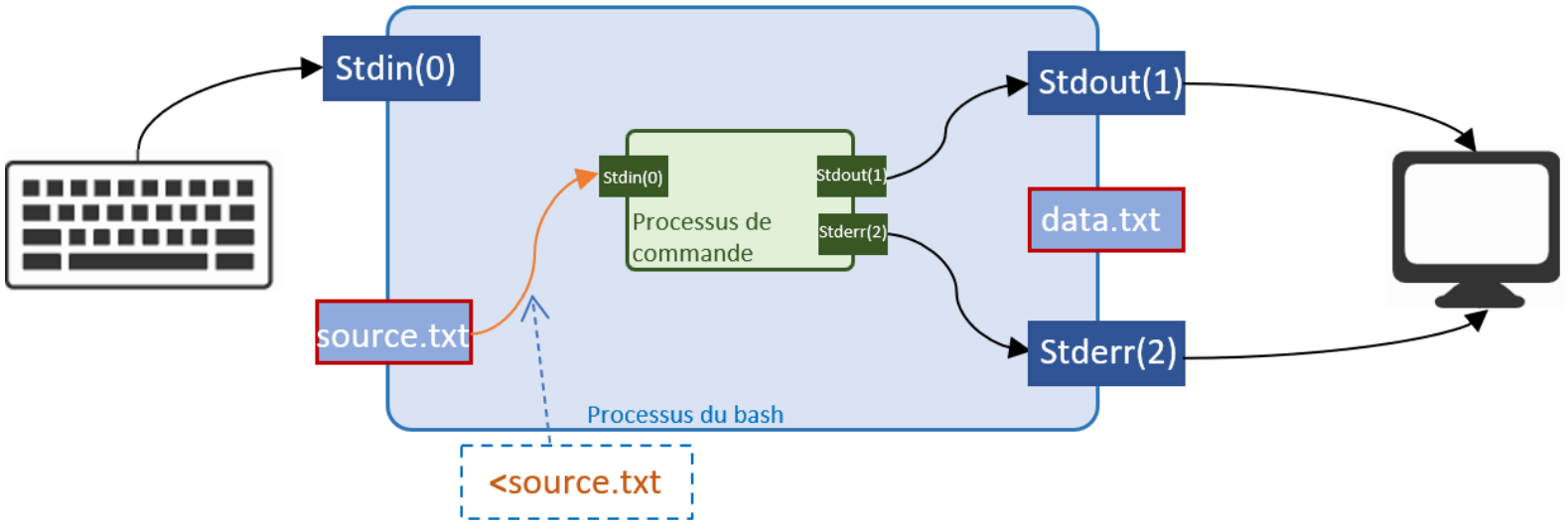
Exemple :

```
ali@DESKTOP-A3SUPDN:~$ ls /root 2> erreur.log
ali@DESKTOP-A3SUPDN:~$ cat erreur.log
ls: impossible d'ouvrir le répertoire '/root': Permission non accordée
```

```
ali@DESKTOP-A3SUPDN:~$ mkdir /root/tmp 2>>erreur.log
ali@DESKTOP-A3SUPDN:~$ cat erreur.log
ls: impossible d'ouvrir le répertoire '/root': Permission non accordée
mkdir: impossible de créer le répertoire «/root/tmp»: Permission non accordée
```

Redirection de l'entrée standard

La redirection de l'entrée standard concerne toutes les commandes attendant une saisie de la part de l'utilisateur sur le descripteur 0 (saisie clavier).
L'opérateur $<$ permet de rediriger le contenu d'un fichier sur l'entrée standard d'une commande

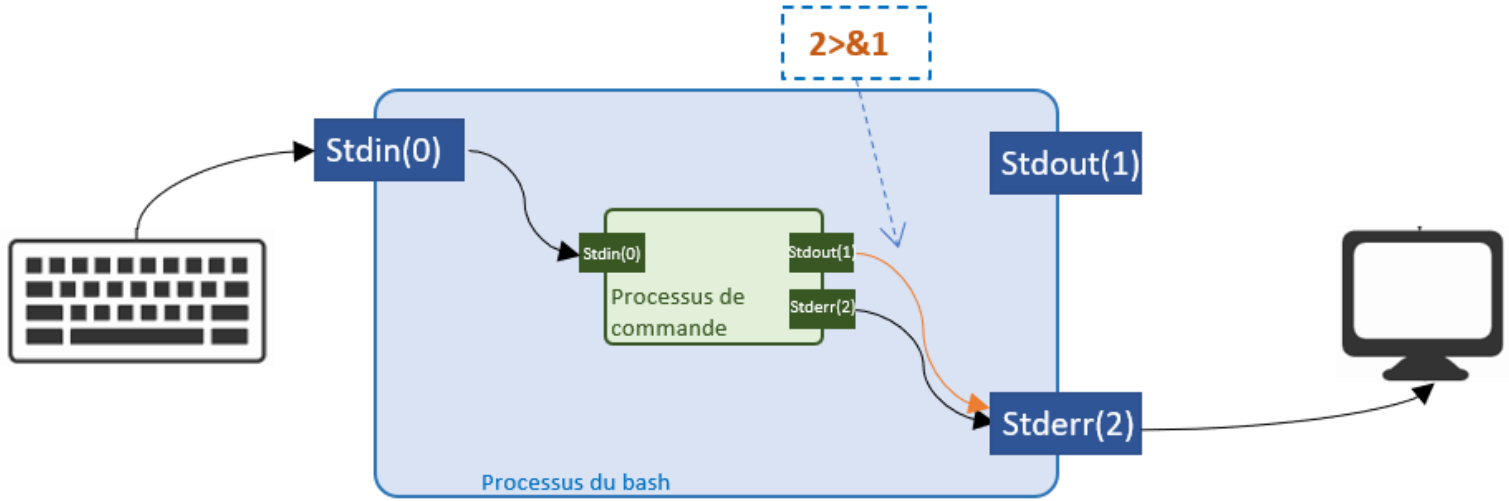


Exemples :

```
ali@DESKTOP-A3SUPDN:~$ echo coucou tous >Bonjour.txt
ali@DESKTOP-A3SUPDN:~$ read x y <Bonjour.txt
ali@DESKTOP-A3SUPDN:~$ echo $x
coucou
ali@DESKTOP-A3SUPDN:~$ echo $y
tous
```

```
ali@DESKTOP-A3SUPDN:~$ wc -l < dromadaire.txt
5
```

Il est parfois utile de rediriger la sortie standard et la sortie d'erreur vers un même endroit.
Pour cela, on utilise le motif `2>&1`



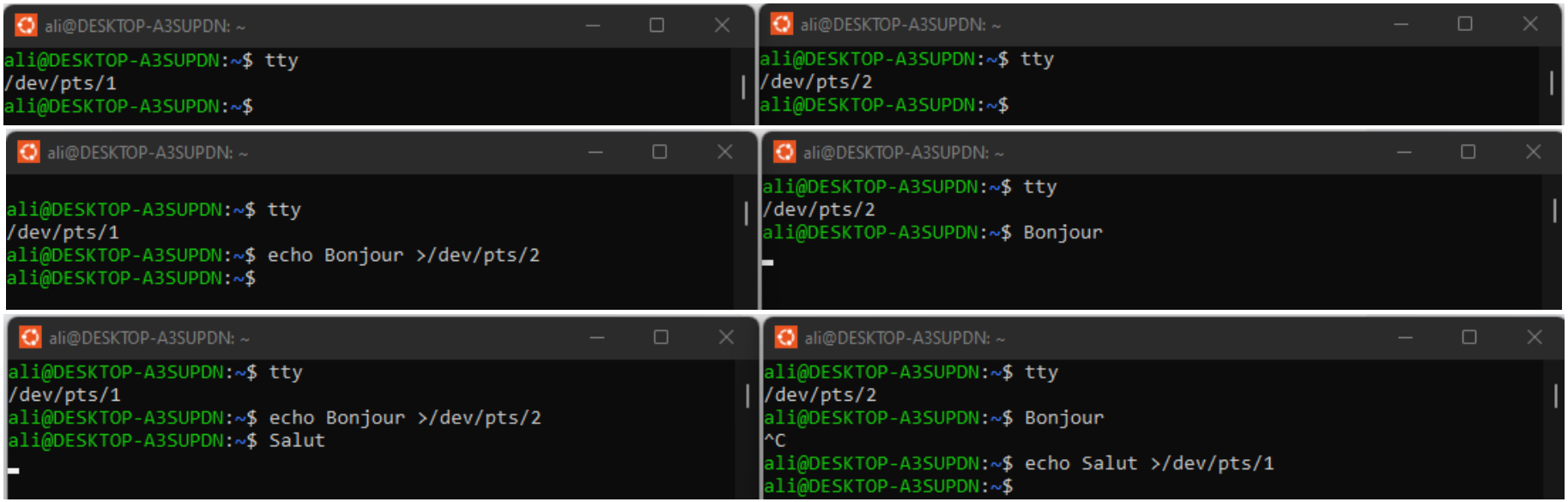
- `>/dev/null 2>&1` : Redirige la sortie d'erreur et la sortie standard vers la poubelle (le périphérique null)
- `>fichier 2>&1` : Redirige la sortie d'erreur et la sortie standard vers fichier
- `>>fichier 2>&1` : Redirige la sortie d'erreur et la sortie standard à la fin de fichier

Autres exemples de redirections

Exemple :

```
ali@DESKTOP-A3SUPDN:~$ (echo Voici une phrase ; date ; jj)
Voici une phrase
sam. 20 août 2022 01:44:34 CEST
jj : commande introuvable
ali@DESKTOP-A3SUPDN:~$ (echo Voici une phrase ; date ; jj) >/dev/null
jj : commande introuvable
ali@DESKTOP-A3SUPDN:~$ (echo Voici une phrase ; date ; jj) >/dev/null 2>&1
ali@DESKTOP-A3SUPDN:~$
```

Exemple : échange/redirection entre deux terminaux

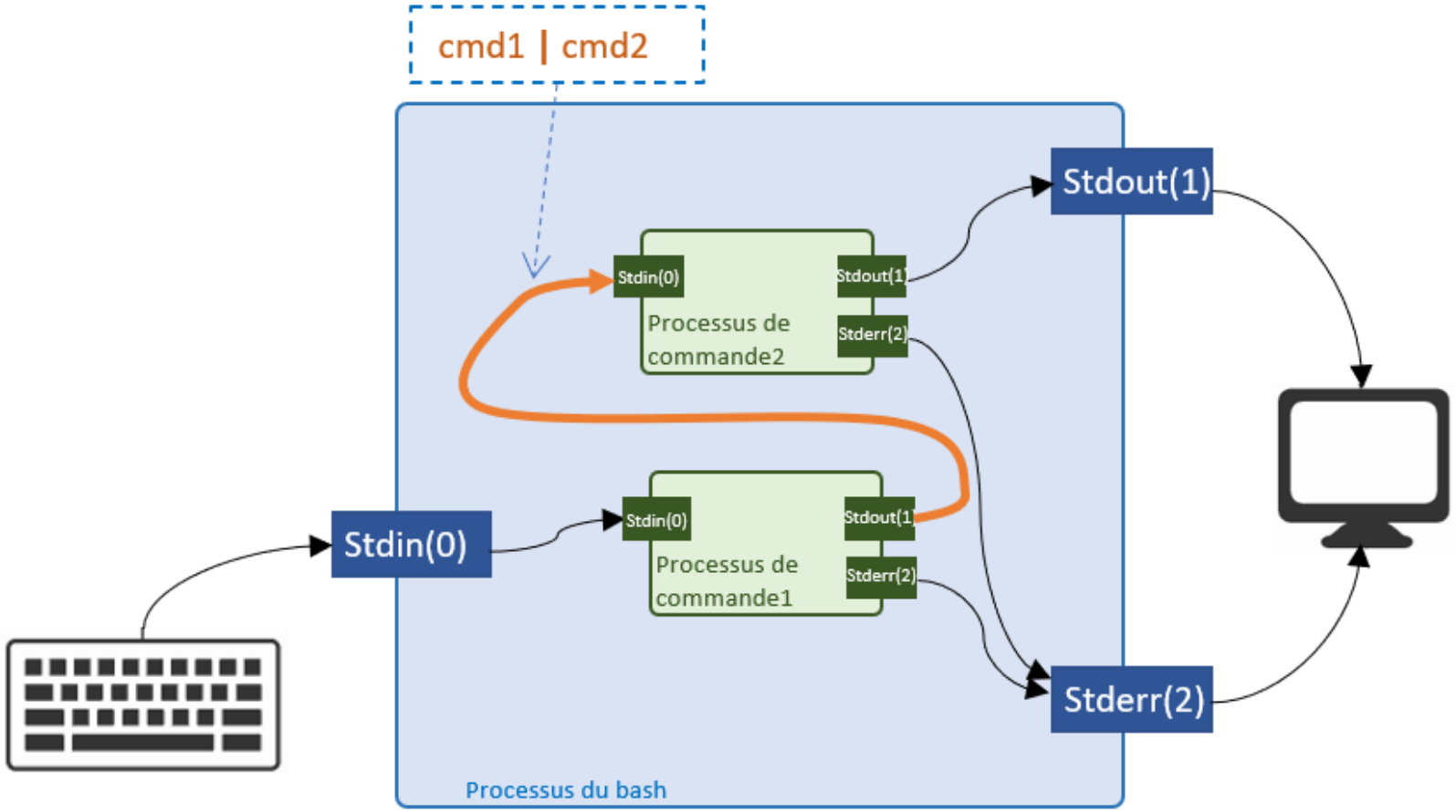


Les tubes

Le symbole | (**tube** ou **pipe**) indique que l'on souhaite rediriger la sortie standard de la commande 1 vers l'entrée standard de la commande 2

Syntaxe : cmd1 | cmd2

- Exécute cmd1 et cmd2 en **parallèle**
- La sortie de cmd1 est redirigée dans l'entrée de cmd2



Les tubes : exemples

Exemple :

```
ali@DESKTOP-A3SUPDN:~/Tubes$ ls |sort -r
Marcel
Luc
Jean
Hugo
Emile
Albert
```

```
ali@DESKTOP-A3SUPDN:~$ cat dromadaire.txt
Avec ses quatre dromadaires
Don Pedro d'Alfaroubeira
Courut le monde et l'admira.
Il fit ce que je voudrais faire
Si j'avais quatre dromadaires.
ali@DESKTOP-A3SUPDN:~$ cat dromadaire.txt | grep Pedro
Don Pedro d'Alfaroubeira
ali@DESKTOP-A3SUPDN:~$
```

Autres exemples

La commande tee divise en (T) la sortie d'une commande.

On peut afficher le résultat d'une commande sur la sortie standard (**écran**) **et** dans un **fichier**.

Exemple :

```
ali@DESKTOP-A3SUPDN:~$ echo Bonjour... | tee fichier1.txt
Bonjour...
ali@DESKTOP-A3SUPDN:~$ cat fichier1.txt
Bonjour...
```

Tableau récapitulatif sur les redirections

Commande	Définition
`C > f`	Exécute la commande "C" et redirige sa sortie standard dans le fichier "*f*" en écrasant son contenu ou en le créant s'il n'existe pas
`C >> f`	Exécute la commande `C` et redirige sa sortie standard à la fin du fichier *f*
`C 2> f`	Exécute la commande `C` et redirige sa sortie d'erreur standard vers le fichier f en écrasant son contenu ou en créant *f* s'il n'existe pas
`C 2>> f`	Exécute la commande `C` et redirige sa sortie standard des erreurs à la fin de *f*
`C 2>&1`	Exécute la commande `C` et redirige sa sortie standard des erreurs dans sa sortie standard
`C < f`	Exécute la commande `C` en passant le contenu de *f* dans son entrée standard
`C1 | C2`	Exécute la commande `C1` et injecte son résultat (le contenu de sa sortie standard) dans l'entrée standard de la commande `C2`
`C |tee f`	lire depuis l'entrée standard et écrire sur la sortie standard tout en écrivant dans des fichiers

Plan

1 Flux de redirections

2 Processus

3 Commandes incontournables

Notion de processus

- **Processus** = programme en cours d'exécution (n'importe quel exécutable exécuté)
  Si le *processus 2* a été lancé par le *processus 1*, on l'appelle un processus **fils**. Le processus qui l'a lancé est appelé processus **parent**.

- **Caractéristiques** statiques d'un processus :
  - *PID* : Process Identifier (identifie le processus)
  - *PPID* : Parent Processus Identifier (identifie le parent)
  - *Utilisateur propriétaire*
  - *Droits d'accès aux ressources* (fichiers, etc.)

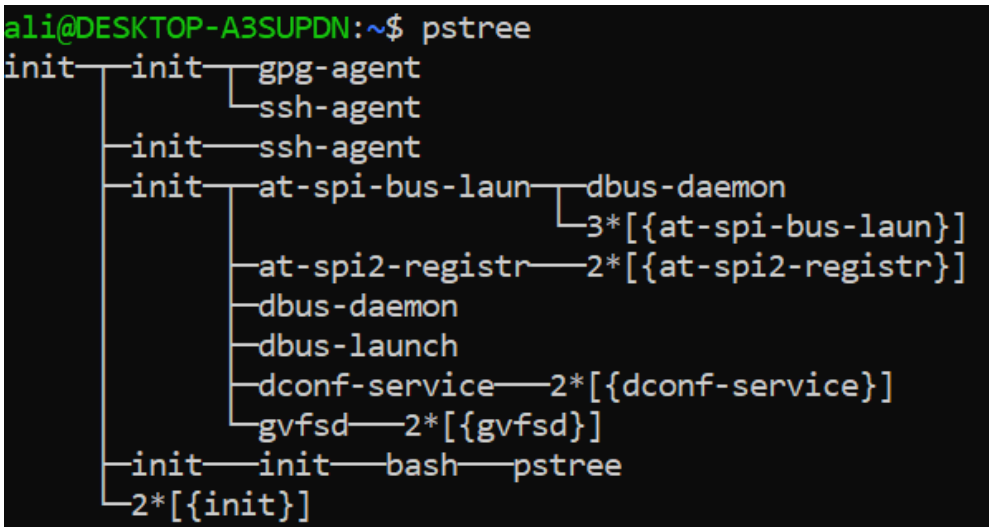
- **Caractéristiques** dynamiques d'un processus :
  - Priorité, environnement d'exécution, etc.
  - *PPID* : Parent Processus Identifier (identifie le parent)
  - Quantité de ressources consommées (temps CPU, etc.)

L'arborescence des processus

Commande : pstree affiche la **hiérarchie** des processus **parents** et **fils**.

Options : -p pour afficher les PIDs
-h pour ressortir (en gras) les processus utilisateurs.

Exemple :



- Chaque processus possède un processus **parent**
  - Sauf le premier processus (**systemd** ou **init**, **PID=1**)
    ⇒ arborescence de processus

Observation des processus

Commande : La commande ps pour lister les processus.
Sans aucune option, ps n'affiche pas la liste complète des processus, ni beaucoup d'informations.

Syntaxe : ps [options]

Options : aux affiche **tous** les processus en cours d'exécution
- a – Processus de **tous les utilisateurs**
- u – Afficher le format orienté utilisateur (plus d'infos sur les processus en cours d'exécution)
- x – Liste des processus qui n'appartiennent à aucune TTY
- PID – affiche les information du processus numéro PID

Exemples :

```
ali@DESKTOP-A3SUPDN:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          32  0.0  0.0   2112   352 ?        S    août19   0:00 /init
ali           85  0.0  0.0   6036   452 ?        Ss   août19   0:00 /usr/bin/ssh-agent
root         374  0.0  0.0   2120   360 ?        S    août19   0:00 /init
ali          416  0.0  0.0   6036   456 ?        Ss   août19   0:00 /usr/bin/ssh-agent
root       15620  0.0  0.0   2252   492 ?        R    11:59   0:00 /init
ali        15621  0.0  0.0  11808  6292 pts/1    Ss   11:59   0:00 -bash
root       16098  0.1  0.0   2252   492 ?        S    15:06   0:00 /init
ali        16099  1.5  0.0  10828  5204 pts/2    Ss   15:06   0:00 -bash
ali        16114  0.6  0.0  16176  4544 pts/2    S+   15:06   0:00 xeyes
ali        16115  0.0  0.0  11532  3432 pts/1    R+   15:06   0:00 ps aux
```

```
ali@DESKTOP-A3SUPDN:~$ ps 16114
  PID TTY      STAT   TIME COMMAND
16114 pts/2    S+     0:01 xeyes
```

Observation des processus

Commande : La commande top affiche les processus consommant le plus de ressource CPU, en rafraichissant l'affichage régulièrement.
Permet de surveiller son système et d'identifier des dysfonctionnements.
Pour quitter top, tapez q ou Ctrl+c.
Exemple :

```
top - 15:16:15 up 1 day, 12:54,  0 users,  load average: 0,08, 0,02, 0,01
Tâches:  22 total,   1 en cours,  21 en veille,   0 arrêté,   0 zombie
%Cpu(s):  0,1 ut,  0,0 sy,  0,0 ni, 99,9 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
MiB Mem :   7884,5 total,   6327,8 libr,    117,5 util,   1439,2 tamp/cache
MiB Éch:   2048,0 total,   2046,7 libr,      1,3 util.   7496,4 dispo Mem

  PID UTIL.      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TEMPS+ COM.
16114 ali        20   0   16176   4544   4124 S   0,7   0,1   0:02.89 xeyes
    1 root       20   0    1904   1096   1020 S   0,0   0,0   0:15.55 init
   32 root       20   0    2112    352      0 S   0,0   0,0   0:00.06 init
   85 ali        20   0    6036    452      0 S   0,0   0,0   0:00.00 ssh-agent
   88 ali        20   0   82952    916    624 S   0,0   0,0   0:00.00 gpg-agent
  374 root       20   0    2120    360      0 S   0,0   0,0   0:00.00 init
  416 ali        20   0    6036    456      0 S   0,0   0,0   0:00.00 ssh-agent
14559 root       20   0    2180    420      0 S   0,0   0,0   0:00.04 init
14942 ali        20   0    7032   2392   2012 S   0,0   0,0   0:00.00 dbus-launch
14943 ali        20   0    7216   2976   2484 S   0,0   0,0   0:00.03 dbus-daemon
14947 ali        20   0  305360   8484   5888 S   0,0   0,1   0:00.02 at-spi-bus-laun
14952 ali        20   0    7108   3752   3416 S   0,0   0,0   0:00.03 dbus-daemon
14955 ali        20   0  156224   5528   4936 S   0,0   0,1   0:00.02 dconf-service
14961 ali        20   0  162868   6304   5716 S   0,0   0,1   0:02.85 at-spi2-registr
15242 ali        20   0  240016   7880   7056 S   0,0   0,1   0:00.02 gvfsd
15619 root       20   0    2252    492      0 S   0,0   0,0   0:00.00 init
15620 root       20   0    2252    492      0 S   0,0   0,0   0:00.36 init
15621 ali        20   0   11808   6292   3940 S   0,0   0,1   0:00.86 bash
```

Variables relatives aux processus

- Chaque processus *bash*, y compris les scripts, définissent :
  - $$: PID du bash courant
  - $PPID : PID du parent du bash courant

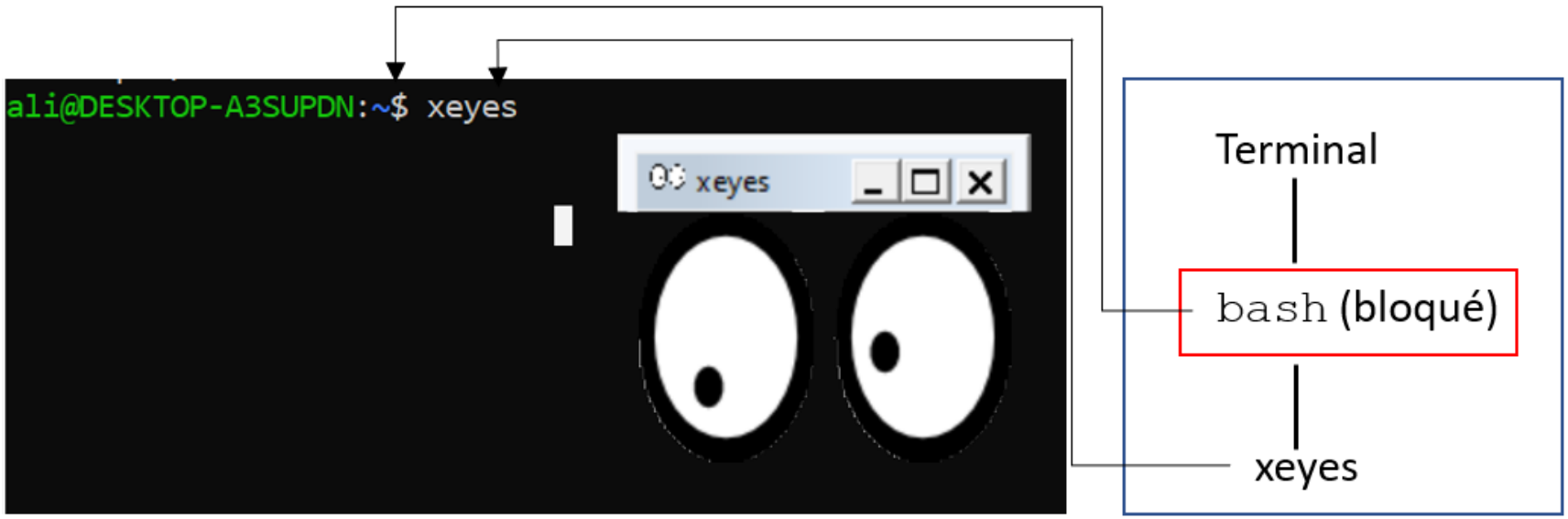
Exemple :

```
ali@DESKTOP-A3SUPDN:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root       15620  0.0  0.0   2252   492 ?        R    11:59   0:00 /init
ali        15621  0.0  0.0  11808  6292 pts/1    Ss   11:59   0:00 -bash
root       16097  0.0  0.0   2252   492 ?        Ss   15:06   0:00 /init
root       16098  0.0  0.0   2252   492 ?        S    15:06   0:00 /init
ali        16099  0.0  0.0  10828  5204 pts/2    Ss   15:06   0:00 -bash
ali        16114  0.4  0.0  16176  4544 pts/2    S+   15:06   0:04 xeyes
ali        16138  0.0  0.0  11532  3372 pts/1    R+   15:22   0:00 ps aux
ali@DESKTOP-A3SUPDN:~$ echo $$
15621
ali@DESKTOP-A3SUPDN:~$ echo $PPID
15620
ali@DESKTOP-A3SUPDN:~$ ps -p 15621,15620
  PID TTY          TIME CMD
15620 ?        00:00:00 init
15621 pts/1    00:00:00 bash
```

Processus en avant-plan

- Par défaut, une commande s'exécute en avant-plan (foreground)
  - bash crée un processus **fils** et **attend** qu'il se termine
  - Le processus fils exécute le programme

Exemple :



bash est **bloqué** tant que le processus fils (xeyes) s'exécute



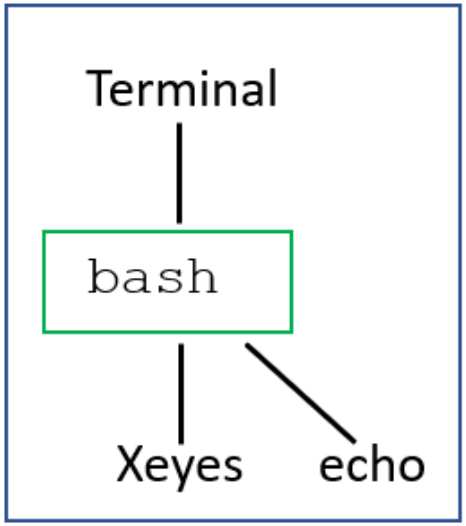
Quand le processus fils (xeyes) se termine, bash reprend son exécution

Processus en arrière-plan

- Pour exécuter une commande en arrière-plan (background), on termine la commande par &
  - `bash` crée un processus **fils** et **n'attend pas** qu'il se termine
  - `bash` affiche le numéro de job (JobID) et le PID du fils
  - Le processus fils exécute le programme

Reprenons le même exemple :





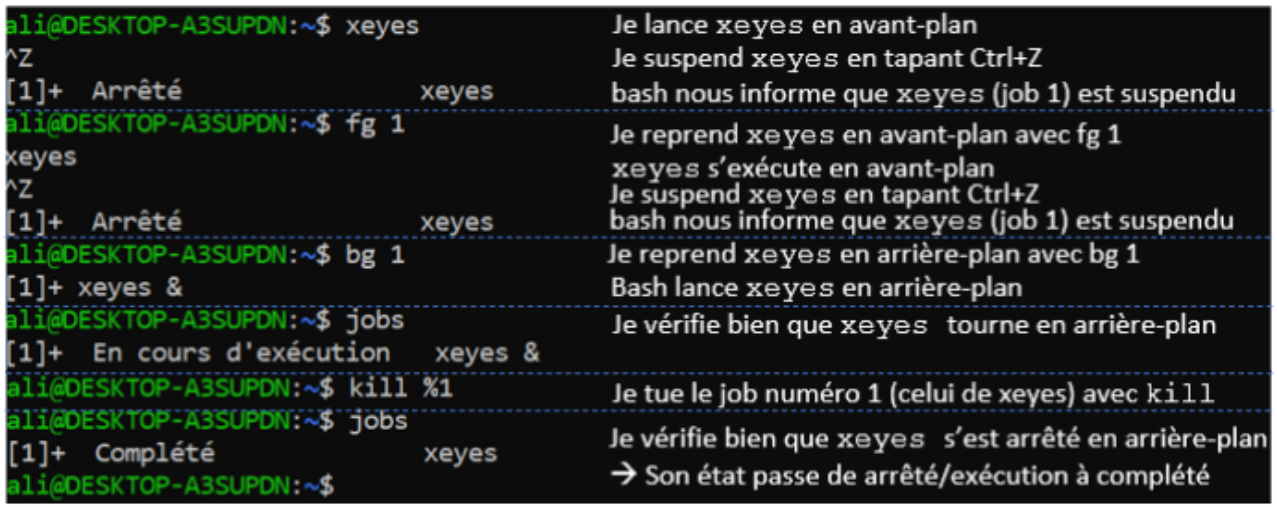
`bash` et le processus fils (`xeyes`) s'exécute en **parallèle**
`bash` peut donc exécuter **d'autres** commandes (`echo $$`)..
Quand le fils (xeyes) se termine, le SE informe `bash` (`[1] + Done xeyes`)

Remarque : Le **PID** du dernier processus lancé en arrière-plan est dans la variable $!

Suspendre un processus

- Un processus peut être suspendu en tapant CTRL+Z pendant son exécution.
  - Le *bash* reprend immédiatement la main.
  - Le processus est "endormi", mais toujours "vivant" : son exécution peut être reprise.
- Reprendre un processus en attente (suspendu)
  - Pour le mettre en avant-plan : fg (**foreground**)
    Syntaxe : fg N: mettre en avant-plan le job N
  - Pour le mettre en arrière-plan : bg (**background**)
    Syntaxe : bg N: mettre en arrière-plan le job N

Exemple :



Suppression d'un processus

Un processus se termine :

- s'il atteint sa dernière instruction
- ou, s'il appelle `exit`
- ou, s'il reçoit un `signal`
  - `control-c` : tue le processus en avant plan
  - `kill` ou `killall` : tue un processus
    - `kill %JobID` : tue le processus de numéro de job *JobID*
    - `kill PID` : tue le processus d'identifiant *PID*
    - `killall prog` : tue tous les processus dont le chemin du programme est *prog*

Remarque : les processus peuvent résister à `control-c`, `kill` ou `killall`.
Si c'est le cas, ajoutez `-9` (`SIGKILL`) après `kill`/`killlall` pour forcer leur mort
Un signal est un message envoyé par le SE à un processus.

Exemples : `SIGTERM` : demande poliment au processus de s'arrêter
`SIGKILL` : tue le processus (lorsque `SIGTERM` échoue)
`SIGTSTP` : suspend le processus (c'est le `Ctrl+Z`)
`SIGCONT` : relance un processus suspendu (c'est `bg` et `fg`)

Commandes utiles pour les processus

- `Ctrl-C` : arrête l'exécution en cours (avant-plan)
- `Ctrl-Z` : suspend l'exécution en cours (avant-plan) et redonne la main à la console (place le processus en attente)
- `Ctrl-D` : termine la saisie (ferme le terminal)
- `commande &` : exécute la commande en arrière-plan
- `jobs` : liste des commandes en arrière-plan (options **-l** et **-p**)
- `bg` ou `bg %<num>` : reprend et bascule le `job` donné en arrière-plan (si on a oublié `&`, faire `Ctrl-Z` puis `bg`)
- `fg` ou `fg %<num>` : reprend en avant-plan l'exécution suspendue ou en arrière-plan donnée
- `kill %<num>` : termine (poliment) l'exécution du `job` *num*
- `Ctrl-L` : rafraichit l'écran (`clear`, `reset`)

Enchaînement

L'enchaînement est le lancement **plusieurs** processus (commandes) depuis **une seule ligne de commande**.
Plusieurs moyens :

- **Enchaînement parallèle** avec redirections via des tubes
  Syntaxe: cmd1 | cmd2 | cmd3
- **Enchaînement séquentiel** : lancer plusieurs processus l'un à la suite des autres, en les séparant par un point-virgule (;)
  Le point-virgule joue exactement le même rôle que le retour à la ligne.
  Syntaxe: cmd1 ; cmd2 ; cmd3
- **Enchaînements conditionnels**
  - "**et alors**" cmd1 && cmd2: la commande de droite est exécutée si et seulement si la commande de gauche a réussi (status $= 0$)
  - "**ou sinon**" cmd1 || cmd2: la commande de droite est exécutée si et seulement si la commande de gauche a échoué (status $\neq 0$)

Remarque : La commande echo $? affiche le **code de status** de la dernière commande exécutée.
Par convention, elle indique le **succès** ($=0$) ou l'**échec** ($\neq 0$) du processus

Plan

1 Flux de redirections

2 Processus

3 Commandes incontournables

Avertissement

Sélection non-exhaustive des commandes les plus utiles

Seules les options les plus utiles sont présentées

Pour aller plus loin, consulter le manuel !

Beaucoup de commandes, sans argument, sont des filtres

- travaillent sur les données lues sur l'entrée standard
- écrivent leur résultat sur la sortie standard

On peut les combiner facilement avec un *pipe* |

`tr`, `head`, `tail`, `cut`, `grep`, `sed`, et beaucoup d'autres ...

Quand vous écrirez vos propres commandes, pensez à faire de même.

Archivage

- Commande `tar` (*tape archive*) pour manipuler des archives
  Archive = rassemblement d'une arborescence de fichiers en un seul fichier

  - `tar -czf fic.tgz rep` : crée l'archive `fic.tgz` à partir de *rep*
  - `tar -xf fic.tgz` : extrait l'archive `fic.tgz`
  - `tar -tf fic.tgz` : liste le contenu de l'archive `fic.tgz`

  Options :

  - `-c chem`, pour créer l'archive à partir du chemin *chem*
  - `-v`, pour un affichage en mode verbeux
  - `-z`, pour une compression des données au format *gzip*
  - `-f nom.tgz`, pour préciser le nom de l'archive voulue
    $\Rightarrow$ Par convention, extension *.tgz* ou *.tar.gz*
  - `-x`, pour extraire (`-z`, pour la décompression via `gzip`)
    $\Rightarrow$ décompression dans le répertoire courant
  - `-t`, pour lister

Recherche de fichiers : `find`

Syntaxe : `find repertoire_de_recherche [Liste des critères]`

recherche des **entrées satisfaisants** un **ensemble de critères de sélection** dans une **arborescence**.
Parcourt **récursivement** et teste les critères sur chaque entrée.

Options :

- `-name "<motif>"` : précise le nom des entrées à rechercher
  `<motif>` est motif conforme à bash à l'exception des crochets [. . .]
  Attention : mettre le motif entre guillemets (")
- `-type <type>` : précise le type des entrées à rechercher
  `f` : fichier normal ; `d` : dossier ; `l` : lien symbolique

Exemples :

`find . -atime 24h` #affiche tous les fichiers ayant été accédés dans les dernières 24h

`find -name "*.c"` #affiche tous les fichiers **C** à partir du répertoire principal

`find TP1 -name "*.o" -delete` #suppression récursive de tous les **.o** à partir du répertoire *TP1*

`find /usr -name "*.c"` #affiche tous les fichiers dont le nom se terminent par **.c** sous **/usr**

Extraire le nom du fichier à partir d'un chemin : basename

Syntaxe : `basename fichier [suffixe]`

La commande `basename` permet d'extraire le nom du fichier à partir d'un chemin.

Affiche `fichier` privée de son chemin (tout ce qui se trouve avant le dernier /) et de son éventuel `suffixe` (si `fichier` se termine par `suffixe`)

permet aussi d'obtenir le dernier segment du chemin passé en argument

Exemple :

```
basename /enseignement/inf203/TD6.pdf
basename /enseignement/inf203/TD6.pdf .pdf
basename /enseignement/inf203/TD6.pdf .c
```

Affiche :

```
TD6.pdf
TD6
TD6.pdf
```

Extraire le chemin parent d'un fichier : dirname

La commande dirname permet d'extraire le chemin parent d'un fichier.

Syntaxe : `dirname /rep`

affiche le nom du répertoire contenant le fichier ou répertoire placé en paramètre

Exemple :

```
dirname /enseignement/S11/TD6.pdf
dirname S11/TD6.pdf
dirname TD6.pdf
```

Affiche :

```
/enseignement/S11
S11
.
```

Remarque : on peut aussi faire :

```
file="/enseignement/S11/TD6.pdf"; echo ${file%/*}
```

Pour les fichiers texte : grep recherche de motif

Syntaxe : `grep [-c | -v] motif [fichier ...]`

affiche les lignes des fichiers qui contiennent le *motif*

Options :

- `-v` : inverse le motif (affiche les lignes qui ne le contiennent pas)
- `-r` : traite récursivement les fichiers dans le dossier passé en argument
- `-i` : ignore la casse

Attention : Le **motif** est une**expression régulière** !
Les méta-caractères de `grep` sont **différents de ceux de** `bash` !
Attention : mettre le **motif entre guillemets simples** (')

Exemples :
Pour l'ensemble des lignes du fichier *IUT.txt* :

```
grep But IUT.txt    #affiche celles contenant But
grep -c But IUT.txt    #affiche le nombre de celles contenant But
grep -v '^[mn]'IUT.txt    #Affiche les lignes ne commençant pas par m ou n
grep -v -c But IUT.txt    #affiche le nombre de celles ne contenant pas But
```

Pour les fichiers texte : tr transformation de caractères

Syntaxe : `tr chaîne1 chaîne2`

`tr -s chaîne`

copie l'entrée standard sur la sortie standard en **substituant**

- les caractères présents dans `chaîne1` par le caractère de position correspondante dans `chaîne2`
- les répétitions des caractères présents dans `chaine` par une occurrence unique

Attention : la commande ne prend pas de fichier en argument - toujours à partir de *stdin* (clavier).

Exemples :

`echo toto | tr o a` #affiche `tata`

`echo maman | tr man yoo` #affiche `yoyoo`

`echo tommme | tr -s m` #affiche `tome`

`cat fic | tr '\n ''ab'` #transforme les retours à la ligne en **a** et les espaces en transforme les retours en **b**

`tr -d 'car' chaine` #élimine chaque caractère `car` de la chaîne

Pour les fichiers texte : `head` et `tail` affichage

Syntaxe : `head [-n nbl | -c nbc] [fichier ...]`

affiche le début de ses entrées

- les `nbl` premières lignes (option par défaut, 10)
- les `nbc` premiers octets

Exemples :

```
head toto.txt    #affiche les 10 premières lignes du fichier toto.txt
head -n 5    #affiche les 5 premières lignes de l'entrée standard
head -c 42 toto.txt    #affiche les 42 premiers caractères du fichier toto.txt
```

De même : `tail [-n nbl | -c nbc] [fichier ...]`

affiche la fin de ses entrées

Pour les fichiers texte : cut extraction de caractères et colonnes

Syntaxe : `cut -c plage fichier`

extrait des caractères de chaque ligne de *fichier* , avec :

- `plage : num` : ou `plage : num1, num2, ...` ou `plage : num1-num2`

et

Syntaxe : `cut -d sep -f plage fichier`

extrait des champs (colonnes) de *fichier*, avec :

- `-d sep : sep` : séparateur de champs (tabulation par défaut)
- `plage` : comme avec `-c`

Exemples :

`cut -c 3-5 toto.txt` #extrait les caractères 3 à 5 de `toto.txt`

`cut -c 5-` #extrait les caractères situés entre la position 5 et la fin de ligne sur l'entrée standard

`cut -f 2-5 toto.txt` #extrait les colonnes 2 à 5 avec la tabulation comme séparateur (par défaut) de colonnes de `toto.txt`

`cut -f 2,4 -d ' ' toto.txt` #extrait les colonnes 2 et 4 avec le caractère ' ' (espace) comme séparateur de colonnes

Pour les fichiers texte : sort tri

Syntaxe : `sort [-n | -k pos1[,pos2]] [fichier ...]`

Trie les lignes lues sur l'entrée standard, et les affiche dans l'ordre sur la sortie standard, avec :

- `-r`, –reverse, pour inverser l'ordre du tri appliqué
- `-n`, –numeric-sort, selon un ordre numérique (plutôt qu'alphabétique)
- `-k`, –key=NUMÉRO, le champ sur lequel trier (les champs sont délimités par tous les types d'espaces)
- `-t`, –field-separator=CARACTÈRE, utilise `CARACTÈRE` pour changer le séparateur (espaces par défaut)

Exemples :

```
sort -k 2,2 fichier    #tri selon le 2ème champ de chaque ligne de fichier
sort -k 2,2 -k 3,3 fic    #tri selon les 2ème et 3ème champs de fic
```

Pour conclure !

- Concepts clés :
  - Flux de redirection, redirection d'entrées/sorties
  - Tubes, enchaînement,
  - Processus, arrière-plan, avant-plan
  - Archivage

- Commandes clés :
  - read, echo, $>$, $>>$, $<$, 2 $>$, 2 $>>$, 2 $>$ &1, |, tee, &&, ||, ;,
  - tty, pstree, ps, top, &, kill, echo $$, echo $PPID, echo $!, echo $?, bg, fg, jobs
  - sort, cut, head, tail, tr, grep
  - `basename, dirname, find, tar`

Merci de votre attention



Exemple

Afficher les 4 plus gros fichiers du répertoire

```
ls -l
```

```
total 1184
-rw-r--r-- 1 zebulon staff 9422 Feb 12 11:20 Slides.aux
-rw-r--r-- 1 zebulon staff 104812 Feb 12 11:20 Slides.log
-rw-r--r-- 1 zebulon staff 6078 Feb 12 11:20 Slides.nav
-rw-r--r-- 1 zebulon staff 1845 Feb 12 11:20 Slides.out
-rw-r--r--@ 1 zebulon staff 266813 Feb 12 11:20 Slides.pdf
-rw-r--r-- 1 zebulon staff 0 Feb 12 11:20 Slides.snm
-rw-r--r-- 1 zebulon staff 24347 Feb 12 11:27 Slides.tex
-rw-r--r-- 1 zebulon staff 27762 Feb 7 11:35 Slides.tex~
-rw-r--r-- 1 zebulon staff 190 Feb 12 11:20 Slides.toc
-rw-r--r-- 1 zebulon staff 204 Feb 12 11:20 Slides.vrb
-rwxr-xr-x 1 zebulon staff 40 Feb 7 11:35 addition2.sh
-rw-r--r-- 1 zebulon staff 5 Feb 7 11:35 deux_entiers
-rwxr-xr-x 1 zebulon staff 206 Feb 7 11:35 exemple_args.sh
```

Exemple

Afficher les 4 plus gros fichiers du répertoire

```
ls -l | tr -s ' '
```

```
total 1184
-rw-r--r-- 1 zebulon staff 9422 Feb 12 11:20 Slides.aux
-rw-r--r-- 1 zebulon staff 104812 Feb 12 11:20 Slides.log
-rw-r--r-- 1 zebulon staff 6078 Feb 12 11:20 Slides.nav
-rw-r--r-- 1 zebulon staff 1845 Feb 12 11:20 Slides.out
-rw-r--r--@ 1 zebulon staff 266813 Feb 12 11:20 Slides.pdf
-rw-r--r-- 1 zebulon staff 0 Feb 12 11:20 Slides.snm
-rw-r--r-- 1 zebulon staff 24347 Feb 12 11:27 Slides.tex
-rw-r--r-- 1 zebulon staff 27762 Feb 7 11:35 Slides.tex~
-rw-r--r-- 1 zebulon staff 190 Feb 12 11:20 Slides.toc
-rw-r--r-- 1 zebulon staff 204 Feb 12 11:20 Slides.vrb
-rwxr-xr-x 1 zebulon staff 40 Feb 7 11:35 addition2.sh
-rw-r--r-- 1 zebulon staff 5 Feb 7 11:35 deux_entiers
-rwxr-xr-x 1 zebulon staff 206 Feb 7 11:35 exemple_args.sh
```

Exemple

Afficher les 4 plus gros fichiers du répertoire

```
ls -l | tr -s ' ' | cut -d' ' -f 5,9
```

```
    9422 Slides.aux
    104812 Slides.log
    6078 Slides.nav
    1845 Slides.out
    266813 Slides.pdf
    0 Slides.snm
    24347 Slides.tex
    27762 Slides.tex~
    190 Slides.toc
    204 Slides.vrb
    40 addition2.sh
    5 deux_entiers
    206 exemple_args.sh
```

Exemple

Afficher les 4 plus gros fichiers du répertoire

```
ls -l | tr -s ' ' | cut -d' ' -f 5,9
```

```
0 Slides.snm
5 deux_entiers
5 un_entier~
40 addition2.sh
190 Slides.toc
204 Slides.vrb
206 exemple_args.sh
1845 Slides.out
6078 Slides.nav
9422 Slides.aux
24347 Slides.tex
27762 Slides.tex~
104812 Slides.log
266813 Slides.pdf
```

Exemple

Afficher les 4 plus gros fichiers du répertoire

```
ls -l | tr -s ' ' | cut -d' ' -f 5,9 | tail -n 4
```

```
24347 Slides.tex
27762 Slides.tex~
104812 Slides.log
266813 Slides.pdf
```

Exemple

Afficher les 4 plus gros fichiers du répertoire

```
ls -l | tr -s ' ' | cut -d' ' -f 5,9 | tail -n 4 | cut -d' ' -f2
```

```
Slides.tex
Slides.tex~
Slides.log
Slides.pdf
```

Exemple : Une solution plus simple ...

Afficher les 4 plus gros fichiers du répertoire

```
ls -S | head -n 4
```

```
    Slides.tex
    Slides.tex~
    Slides.log
    Slides.pdf
```

Remarque : aussi utile, la commande `du` (*disk usage*)

```
du .  | sort -n
```

Rmarque : La commande du (*disk usage*) permet de connaître l'espace utilisé par des fichiers.